

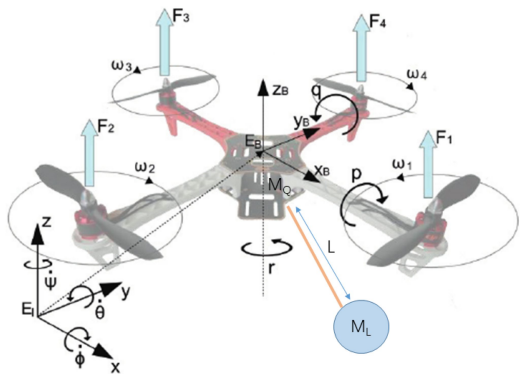
Bridging Control with the Neural Network Verifier

α, β -CROWN: A Tutorial

Tutorial Chairs: Huan Zhang, Bin Hu (UIUC)

Tutorial Paper: <https://arxiv.org/pdf/2605.26577>

Interactive Tutorial & Code: <https://abcrown.org/tutorial>



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

Guarantees for learning-enabled & nonlinear controllers

Many traditional tools are used in nonlinear control:

- Sum-of-squares (SOS), Linear Matrix Inequalities (LMIs), Semidefinite programming (SDP), SMT solvers, ...
- These techniques are broadly applicable to many different problems
- Although possible, it is challenging when dealing with neural network controlled systems

Call for **new tools** for **learning-enabled** / **nonlinear** control systems

- Efficiency and scalability of handling neural networks (NNs) and other nonlinear components
- General enough; potential to handle many different types of problems
- Easy to use

Today's tutorial: α, β -CROWN Neural Network Verification framework as a new tool

Agenda

Part I: Introduction to the verification problem and α, β -CROWN (Huan Zhang)

Part II: A guide on applying α, β -CROWN to control problems (Haoyu Li)

Part III: Interactive coding demo (Xiangru Zhong)

Contributed Talks on more applications in control research

Certifying Learned Safety Certificates: Verification of Neural Barrier and Hamilton–Jacobi Value Functions (Hanjiang Hu, CMU)

Interval methods and monotone systems theory for neural controlled systems (Akash Harapanahalli, Georgia Tech)

Rigorously Analyzing Pixels-to-Torques Policies (Alexander Estornell, Northeastern University)

Bridging Control with the Neural Network Verifier

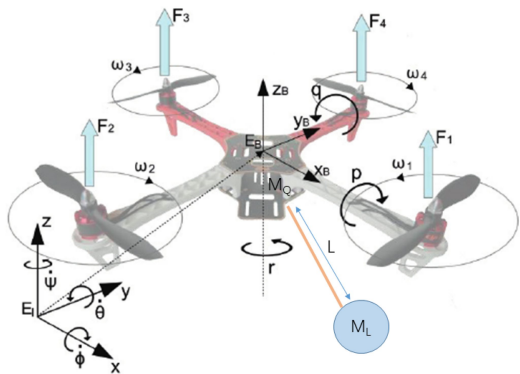
α, β -CROWN: A Tutorial

Part I: Introduction to the verification problem and α, β -CROWN

Huan Zhang (UIUC)

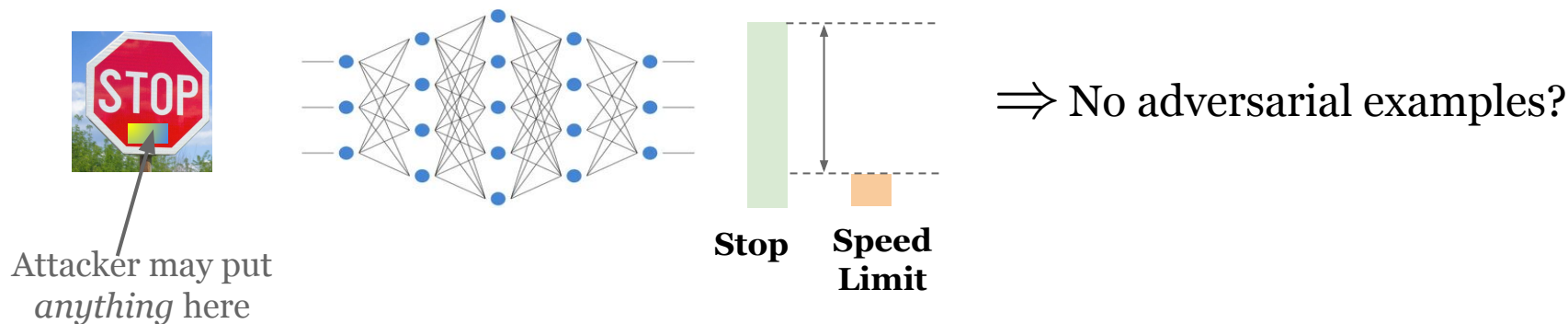
Assistant Professor, Department of Electrical and Computer Engineering

huan@huan-zhang.com



UNIVERSITY OF
ILLINOIS
URBANA - CHAMPAIGN

From NN verification to nonlinear control problems



Neural network verification is originally for formally guaranteeing the safety and robustness of NN classifiers (since 2017)

- **Scalable** to NNs with millions of parameters
- **GPU**-accelerated, orders of magnitudes faster
- But **restricted** to a small class of problems

From NN verification to nonlinear control problems

Today, we use our NN verifier (α,β -CROWN) to solve control problems!

- Give guaranteed **linear bounds** on **complex nonlinear functions** (can be much *tighter* than Lipschitz analysis!)
- Check whether a bunch of **nonlinear inequalities** can be **satisfied** (much *faster* than a SMT solver)
- Solve some **nonlinear optimization problems** in control (your next-gen Gurobi?)

Key characteristics:

- Compose complex nonlinear functions with simple ones
- A neural network is **NOT** required
- Use **GPU** acceleration, great **scalability**



The verification problem

$$f(x)$$

1. A nonlinear function f
 - Can be just the NN itself (e.g., a binary classifier)
 - Can also include NN as part of f (e.g., NN-controlled systems)
 - Can include other nonlinear functions (e.g., dynamical systems + NN)
 - You can compose any supported nonlinear functions (NN is not required), as a computation graph

The verification problem

$$\forall x \in \mathcal{S}, f(x)$$

1. A nonlinear function f
2. A input set $\mathcal{S}$
 - Simple box constraints (coordinate-wise bounds)
 - Convex set (e.g., L_p -norm ball, convex polytopes...)
 - Or non-convex set (e.g., implicitly defined by other constraints)

The verification problem

$$\forall x \in \mathcal{S}, f(x) > 0$$

1. A nonlinear function f
2. A input set S
3. An output specification
 - Simply $f(x) > 0$
 - Complex logic formulas $(f(x) > 0 \vee g(x) > 0) \wedge (h(x) > 0)$

The verification problem

Prove: $\forall x \in \mathcal{S}, f(x) > 0$

(this is a “simple” verification problem in canonical form)

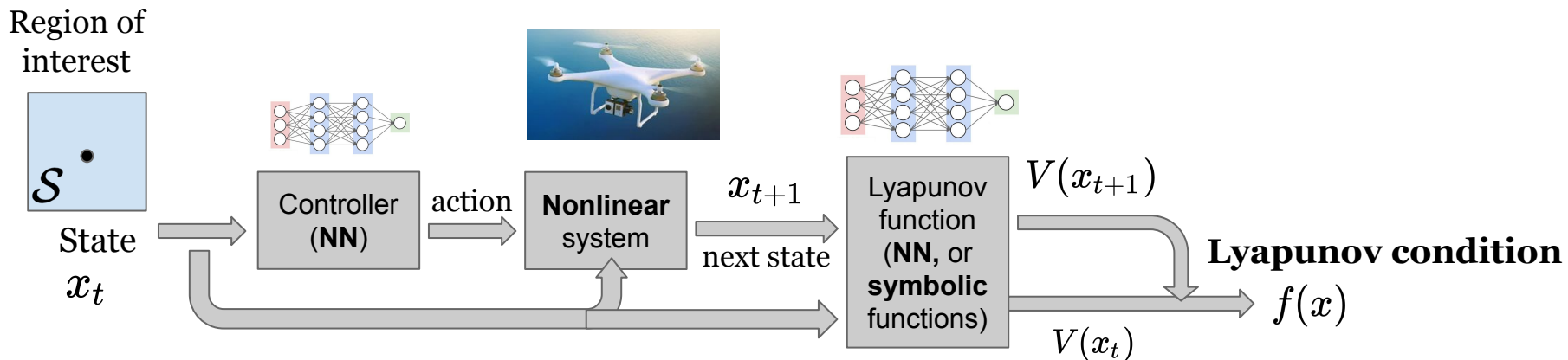
1. A nonlinear function f
2. A input set S
3. An output specification

α, β -CROWN can prove problems like this for you, automatically 

But how can this help control problems ?

Example: verification of the stability of NN controllers

More examples and details in the second part of this tutorial



$$f(x) := V(x_t) - V(x_{t+1}) > 0$$

$f(x)$ is a “computation graph”, representing the algebraic condition you want to verify

Prove: $\forall x \in \mathcal{S}, f(x) > 0$

(other conditions on V can be satisfied by a special construction of V)

Some comparisons to existing tools

Task: Lyapunov stability verification with α, β -CROWN
NN controller + NN Lyapunov function (a few hundreds neurons)

System	dReal	α, β -CROWN
Van der Pol	39265.21s	3.94s
Double Integrator	359.27s	3.00s
Pendulum (large torque)	1479.55s	3.64s
Pendulum (small torque)	1709.19s	3.94s
Path Tracking (large torque)	113.72s	3.77s
Path Tracking (small torque)	142.20s	3.67s

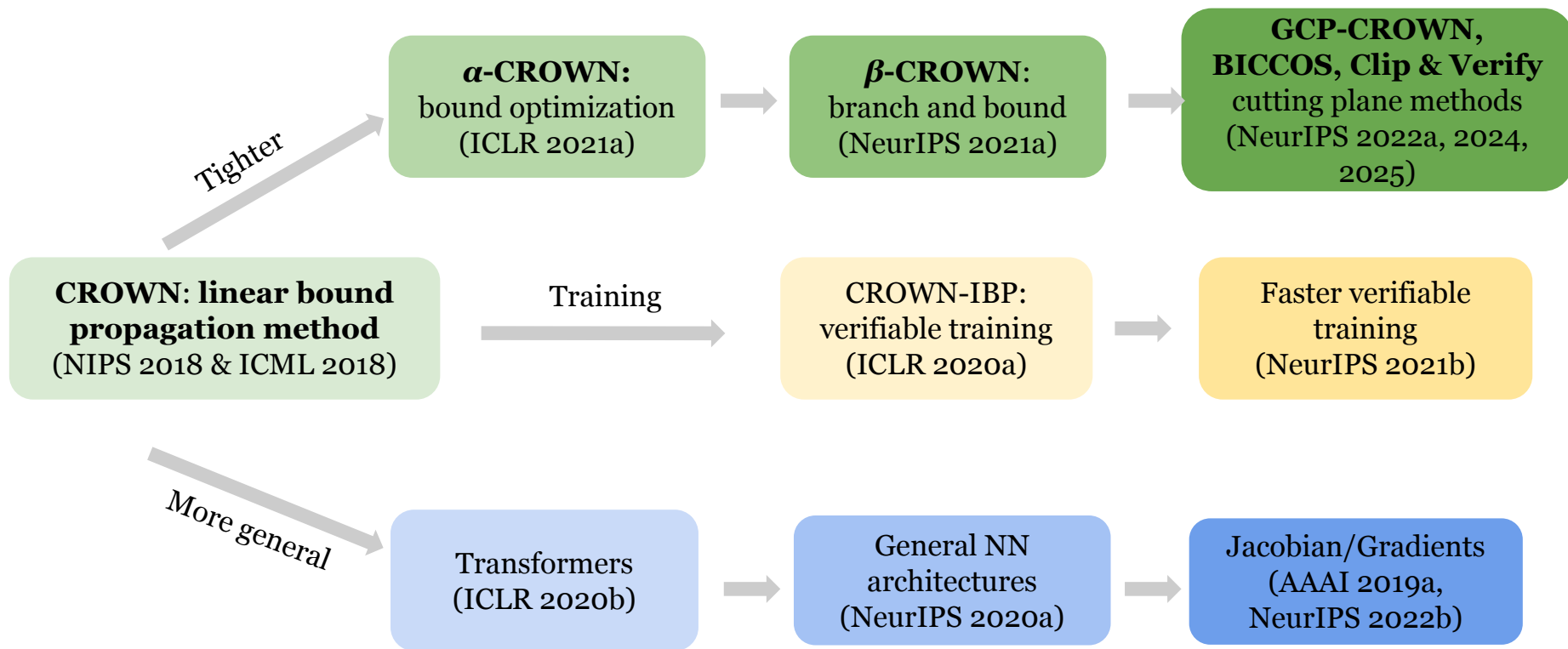
How do we
achieve this?

“Two-Stage Learning of Stabilizing Neural Controllers via Zubov Sampling and Iterative Domain Expansion.” (**Continuous-Time stability analysis**)

Haoyu Li*, Xiangru Zhong*, Bin Hu, Huan Zhang

Paper: <https://arxiv.org/pdf/2506.01356>

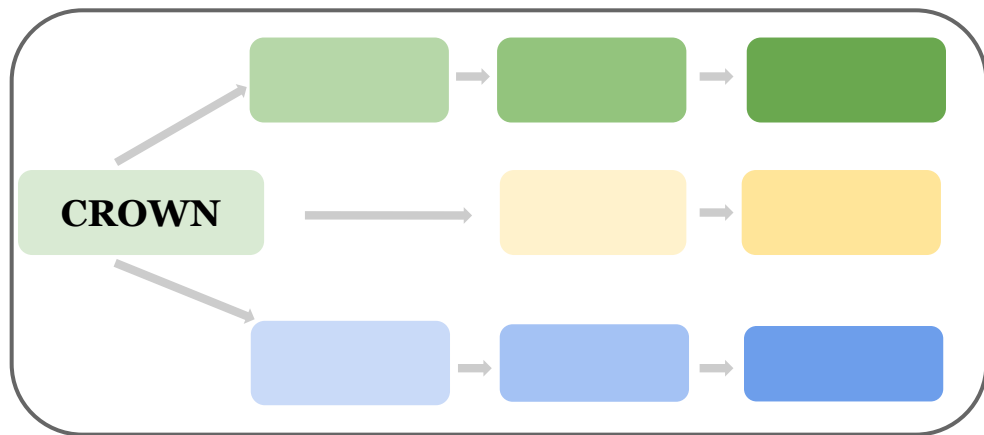
Years of efforts to strengthen the framework



Algorithms to the practical tool: α, β -CROWN

Theoretical framework

Practical verifier: α, β -CROWN

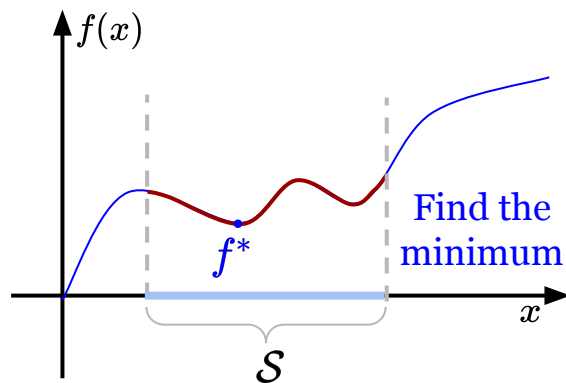


<https://abcrown.org>

Winner of VNN-COMP 2021, 2022, 2023, 2024, 2025

How does α, β -CROWN solve the verification problem?

Prove: $\forall x \in \mathcal{S}, f(x) > 0$



Challenges:

Number of variables $\approx$ number of neurons

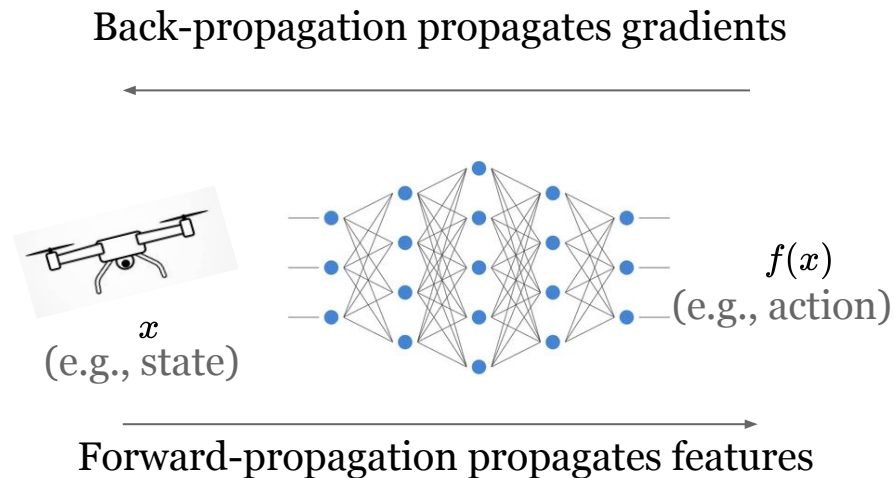
Number of integer variables $\approx$ number of nonlinear functions (e.g., ReLU)

Modern NNs can have **millions** of neurons!

Existing approaches: Linear programming (LP), semidefinite programming (SDP), mixed integer programming (MIP), Satisfiability Modulo Theory (SMT)



We need to exploit the problem structure to be fast!

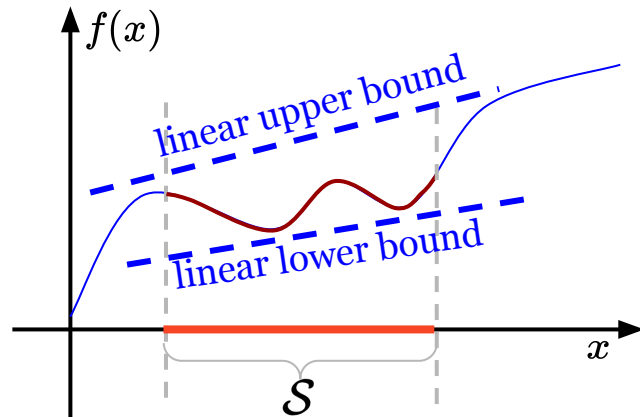
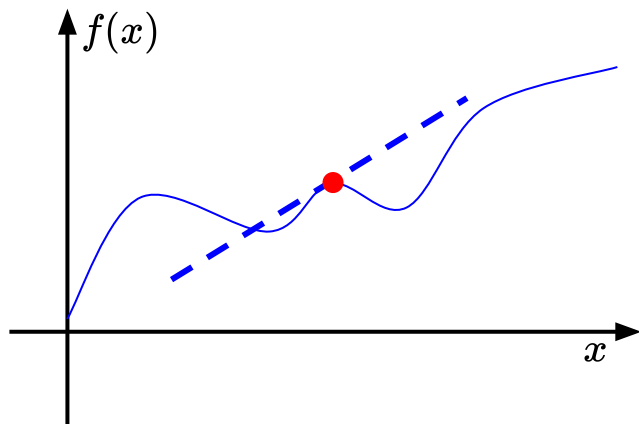


Inspiration: propagating things on NNs and a computation graph is **fast**!

Using off-the-shelf solvers (LP/SDP/MIP/SMT) is **slow**!

We don't want use these; we need special, parallelizable algorithms

Key insight in the α, β -CROWN framework: generalize **backpropagation** to **linear bound propagation**



Gradient is neither an upper nor a lower bound for a non-convex function

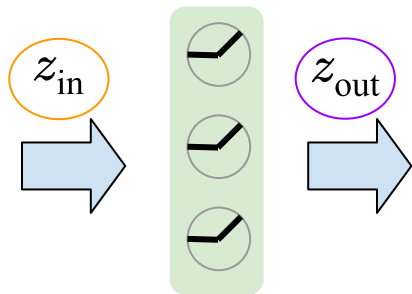
Gradient can be calculated very efficiently
(chain rule & backpropagation)

Can we use linear functions to locally bound nonlinear functions?

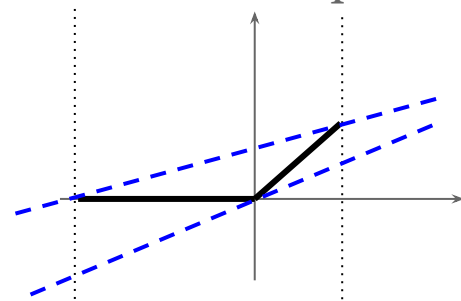
What is the **new “chain rule”** for propagating bounds?

CROWN linear relaxations

Key idea: use linear bounds to relax a non-linear function. Propagation of linear inequalities over relaxed nonlinear functions yields new linear inequalities



$$z_{\text{out}} = \text{ReLU}(z_{\text{in}})$$



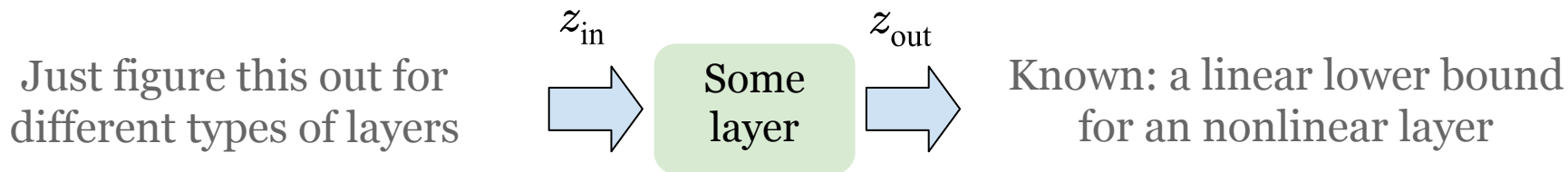
Pre-activation bounds

(can be pre-computed using CROWN)

Theorem (informal): we can efficiently find D , b' such that:

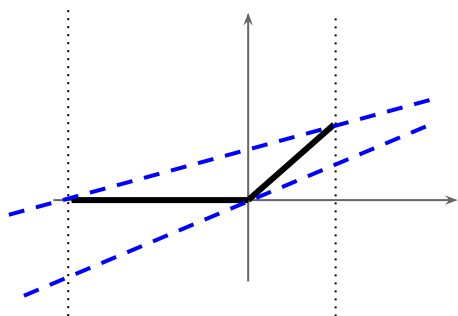
$$f(x) \geq a^\top D z_{\text{in}} + b' \longleftarrow f(x) \geq a^\top z_{\text{out}} + b \quad \forall x \in \mathcal{S}$$

Figure out how to propagate through simple nonlinear functions!

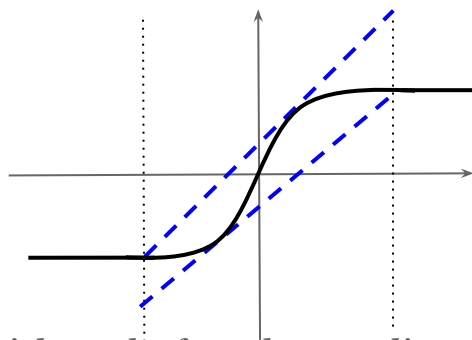
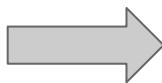


$$f(x) \geq \boxed{a'^{\top}} z_{\text{in}} + \boxed{b'} \quad \longleftarrow \quad f(x) \geq \boxed{a^{\top}} z_{\text{out}} + \boxed{b}$$

Like backpropagation, but we propagate linear inequalities



ReLU

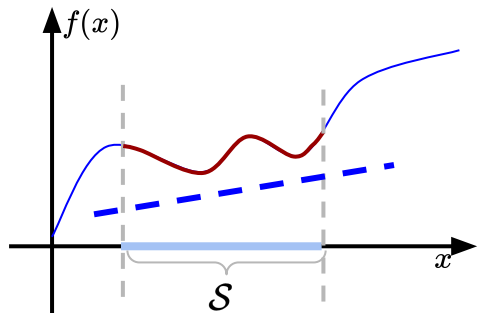


Same trick works for other nonlinear functions

$x^2, x^3, \sqrt{x}, \log x, e^x, |x|, x \cdot y$
 $\sin, \cos, \tan, \tanh, \arctan, \dots$

A local linear bound is
all you need

CROWN: a linear bound propagation algorithm

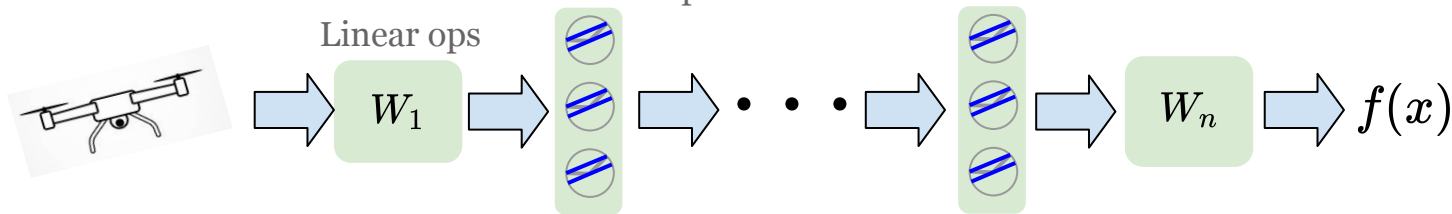


Prove: $\forall x \in \mathcal{S}, f(x) > 0$

Lower bound $> 0 \Rightarrow f(x) > 0 \Rightarrow$ verified

Check out our paper (arxiv.org/pdf/2605.26577) on how we solve other problems

Non-linear ops



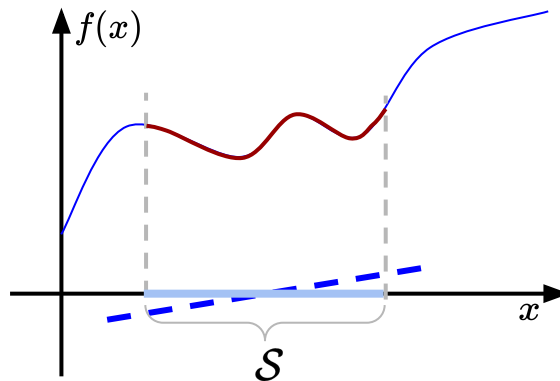
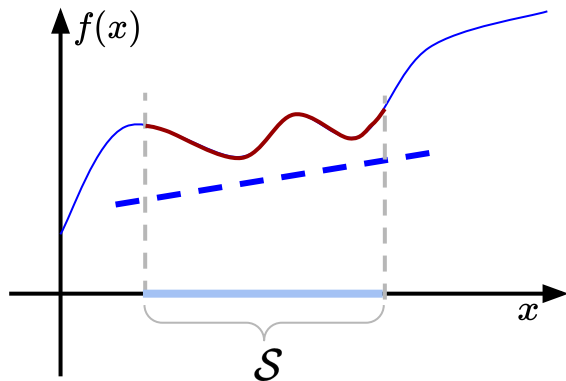
Propagate linear lower bounds (GPU accelerated)

CROWN main theorem (simplified): $f(x) \geq a_{\text{CROWN}}^\top x + b_{\text{CROWN}} \quad \forall x \in \mathcal{S}$

Found by construction (linear bound propagation)

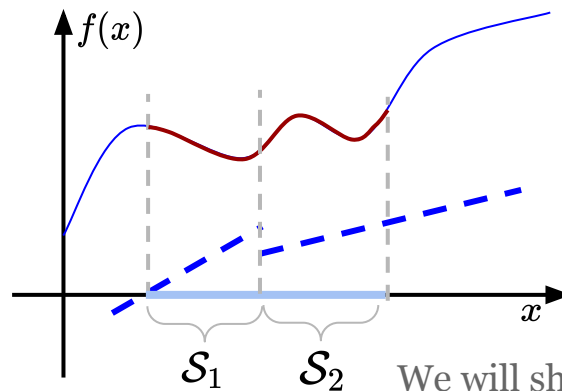
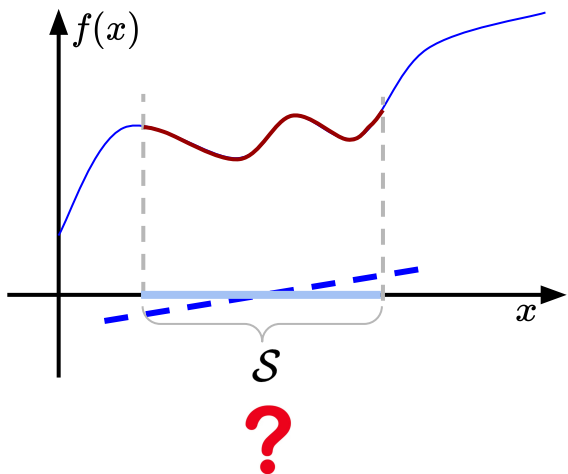
What if the bounds are too loose?

Prove: $\forall x \in \mathcal{S}, f(x) > 0$



Branch and bound to improve tightness

- Branching the original problems into multiple subproblems, each one on a smaller domain with a tighter bound
- Original problem is solved when all subproblems are solved
- α, β -CROWN solves many (e.g., 10,000) subproblems on GPUs in parallel



We will show you these
piecewise linear bound in
coding demo (part 3)

Up next

- Part 2: A guide on applying α, β -CROWN to control problems
 - Reachability analysis, Lyapunov stability, MPC
- Part 3: Interactive coding demo (Colab)
 - α, β -CROWN demonstrated on control problems (with NEW API)
 - A NEW dReal-compatible API - port your code directly!
- Contributed Talks: more involved applications in control
 - Neural barrier & Hamilton-Jacobi functions (CMU), monotone systems theory (Georgia Tech), Pixels-to-Torques policies (Northeastern)
- Interested in more technical backgrounds on α, β -CROWN?
 - Full tutorial paper: <https://arxiv.org/pdf/2605.26577>
 - More tutorial materials at neural-network-verification.com
 - Check out UIUC ECE598HZ course slides